

-- This script was generated by the ERD tool in pgAdmin 4.
-- Please log an issue at <https://github.com/pgadmin-org/pgadmin4/issues/new/choose> if you find any bugs, including reproduction steps.

BEGIN;

```
CREATE TABLE IF NOT EXISTS public.classification
(
    classification_id integer NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1
START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
    lesion_id text COLLATE pg_catalog."default",
    avg_expert_claim text COLLATE pg_catalog."default",
    lesion_type text COLLATE pg_catalog."default",
    expert_accuracy numeric,
    avg_beginner_claim text COLLATE pg_catalog."default",
    beginner_accuracy numeric,
    CONSTRAINT classification_pkey PRIMARY KEY (classification_id)
);
```

```
CREATE TABLE IF NOT EXISTS public.crc_clean
(
    "Crohn's disease" text COLLATE pg_catalog."default",
    "p-value" numeric,
    crc_count numeric,
    crc_percentage numeric,
    "Non-CRC_count" numeric,
    "Non-CRC_percentage" numeric
);
```

```
CREATE TABLE IF NOT EXISTS public.df_final
(
    lesion_id text COLLATE pg_catalog."default" NOT NULL,
    "Type" numeric,
    "Quality_Metric" numeric,
    "Numeric_Value" numeric,
    "Lesion_Type" character varying(20) COLLATE pg_catalog."default",
    CONSTRAINT df_final_pkey PRIMARY KEY (lesion_id)
);
```

```
CREATE TABLE IF NOT EXISTS public.diagnosis
(
    lesion_type text COLLATE pg_catalog."default",
    type numeric,
    lesion_id text COLLATE pg_catalog."default",
```

```

diagnosis_id integer NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1
START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
site text COLLATE pg_catalog."default",
"size [mm]" integer,
CONSTRAINT diagnosis_pkey PRIMARY KEY (diagnosis_id)
);

```

```

CREATE TABLE IF NOT EXISTS public.expertise
(
    expert_id text COLLATE pg_catalog."default" NOT NULL DEFAULT
nextval('expertise_expert_seq'::regclass),
    claim text COLLATE pg_catalog."default",
    lesion_id text COLLATE pg_catalog."default",
    resection text COLLATE pg_catalog."default"
);

```

```

CREATE TABLE IF NOT EXISTS public.gt_clean
(
    lesion_id text COLLATE pg_catalog."default" NOT NULL,
    "GROUND TRUTH" character varying(25) COLLATE pg_catalog."default",
    "EXPERT_1" character varying(25) COLLATE pg_catalog."default",
    "EXPERT_2" character varying(25) COLLATE pg_catalog."default",
    "EXPERT_3" character varying(25) COLLATE pg_catalog."default",
    "EXPERT_4" character varying(25) COLLATE pg_catalog."default",
    "BEGINNER_1" character varying(25) COLLATE pg_catalog."default",
    "BEGINNER_2" character varying(25) COLLATE pg_catalog."default",
    "BEGINNER_3" character varying(25) COLLATE pg_catalog."default",
    "BINARY_EXPERT_1" character varying(25) COLLATE pg_catalog."default",
    "BINARY_EXPERT_2" character varying(25) COLLATE pg_catalog."default",
    "BINARY_EXPERT_3" character varying(25) COLLATE pg_catalog."default",
    "BINARY_EXPERT_4" character varying(25) COLLATE pg_catalog."default",
    "BINARY_BEGINNER_1" character varying(25) COLLATE pg_catalog."default",
    "BINARY_BEGINNER_2" character varying(25) COLLATE pg_catalog."default",
    "BINARY_BEGINNER_3" character varying(25) COLLATE pg_catalog."default",
    CONSTRAINT gt_clean_pkey PRIMARY KEY (lesion_id)
);

```

```

CREATE TABLE IF NOT EXISTS public.lesion
(
    lesion_id text COLLATE pg_catalog."default" NOT NULL,
    ground_truth text COLLATE pg_catalog."default",
    unique_object_id text COLLATE pg_catalog."default",
    CONSTRAINT lesion_pkey PRIMARY KEY (lesion_id)
);

```

```
CREATE TABLE IF NOT EXISTS public.lesion_info
(
    unique_object_id text COLLATE pg_catalog."default" NOT NULL,
    unique_video_name text COLLATE pg_catalog."default",
    "size [mm]" numeric,
    site character varying(50) COLLATE pg_catalog."default",
    histology_extended character varying(50) COLLATE pg_catalog."default",
    histology_class character varying(10) COLLATE pg_catalog."default",
    CONSTRAINT lesion_info_pkey PRIMARY KEY (unique_object_id)
);
```

```
CREATE TABLE IF NOT EXISTS public.lesion_outcome
(
    outcome_id integer NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1
START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
    lesion_id text COLLATE pg_catalog."default",
    status text COLLATE pg_catalog."default",
    tumor_confirmation boolean,
    final_grade text COLLATE pg_catalog."default",
    histology_class text COLLATE pg_catalog."default",
    CONSTRAINT lesion_outcome_pkey PRIMARY KEY (outcome_id)
);
```

```
CREATE TABLE IF NOT EXISTS public.measurement
(
    measurement_id serial NOT NULL,
    lesion_id text COLLATE pg_catalog."default" NOT NULL,
    metric_name text COLLATE pg_catalog."default" NOT NULL,
    metric_type text COLLATE pg_catalog."default",
    metric_value numeric,
    created timestamp without time zone DEFAULT now(),
    CONSTRAINT measurement_pkey PRIMARY KEY (measurement_id)
);
```

```
ALTER TABLE IF EXISTS public.classification
    ADD CONSTRAINT classification_lesion_id_fkey FOREIGN KEY (lesion_id)
    REFERENCES public.lesion (lesion_id) MATCH SIMPLE
    ON UPDATE NO ACTION
    ON DELETE NO ACTION
    NOT VALID;
```

```
ALTER TABLE IF EXISTS public.df_final
```

```
ADD FOREIGN KEY (lesion_id)
REFERENCES public.lesion (lesion_id) MATCH SIMPLE
ON UPDATE NO ACTION
ON DELETE NO ACTION
NOT VALID;
```

```
ALTER TABLE IF EXISTS public.diagnosis
ADD CONSTRAINT diagnosis_lesion_id_fkey FOREIGN KEY (lesion_id)
REFERENCES public.lesion (lesion_id) MATCH SIMPLE
ON UPDATE NO ACTION
ON DELETE NO ACTION
NOT VALID;
```

```
ALTER TABLE IF EXISTS public.expertise
ADD CONSTRAINT expertise_lesion_id_fkey FOREIGN KEY (lesion_id)
REFERENCES public.lesion (lesion_id) MATCH SIMPLE
ON UPDATE NO ACTION
ON DELETE NO ACTION;
```

```
ALTER TABLE IF EXISTS public.gt_clean
ADD CONSTRAINT "LESION" FOREIGN KEY (lesion_id)
REFERENCES public.lesion (lesion_id) MATCH SIMPLE
ON UPDATE NO ACTION
ON DELETE NO ACTION
NOT VALID;
```

```
ALTER TABLE IF EXISTS public.lesion_info
ADD FOREIGN KEY (unique_object_id)
REFERENCES public.lesion (unique_object_id) MATCH SIMPLE
ON UPDATE NO ACTION
ON DELETE NO ACTION
NOT VALID;
```

```
ALTER TABLE IF EXISTS public.lesion_outcome
ADD CONSTRAINT lesion_outcome_lesion_id_fkey FOREIGN KEY (lesion_id)
REFERENCES public.lesion (lesion_id) MATCH SIMPLE
ON UPDATE NO ACTION
ON DELETE NO ACTION
NOT VALID;
```

```
ALTER TABLE IF EXISTS public.measurement
  ADD CONSTRAINT measurement_lesion_id_fkey FOREIGN KEY (lesion_id)
  REFERENCES public.lesion (lesion_id) MATCH SIMPLE
  ON UPDATE NO ACTION
  ON DELETE NO ACTION;

END;
```

-- 1. All metadata for a single lesion

```
SELECT * FROM lesion l
LEFT JOIN lesion_info i ON l.lesion_id = i.unique_object_id
LEFT JOIN diagnosis d ON l.lesion_id = d.lesion_id
LEFT JOIN lesion_outcome o ON l.lesion_id = o.lesion_id
WHERE l.lesion_id = 'adenoma_8';
```

-- 2. Expert accuracy for a given lesion

```
SELECT lesion_id, avg_expert_claim, expert_accuracy
FROM classification
WHERE lesion_id = 'serrated_8';
```

-- 3. Measurements associated with a given lesion

```
SELECT metric_name, metric_value
FROM measurement
WHERE lesion_id = 'adenoma_2';
```

-- 4. An overview of lesion classification (this spans an entire table but shows an example of one instance)

```
CREATE OR REPLACE VIEW lesion_summary AS
SELECT
    l.lesion_id, l.ground_truth,
    i."size [mm]", i.site,
    o.tumor_confirmation,
    c.expert_accuracy,
    m2.metric_value AS numeric_value
FROM lesion l
LEFT JOIN lesion_info i ON l.lesion_id = i.unique_object_id
LEFT JOIN lesion_outcome o ON l.lesion_id = o.lesion_id
LEFT JOIN classification c ON l.lesion_id = c.lesion_id
LEFT JOIN measurement m2 ON l.lesion_id = m2.lesion_id
                        AND m2.metric_name = 'Numeric_Value';
```

```
SELECT * FROM lesion_summary
WHERE lesion_id = 'serrated_8';
```

-- 5. Average lesion size by histology

```
SELECT i.histology_extended, ROUND(AVG(i."size [mm]"), 2) AS avg_size_mm
FROM lesion_info i
GROUP BY i.histology_extended
ORDER BY avg_size_mm DESC;
```

-- 6. Expert vs. Beginner accuracy queries (keep in mind that there are 4 experts and 3 beginners)

```
SELECT
    ROUND(AVG(expert_accuracy), 2) AS expert_accuracy,
    ROUND(AVG(beginner_accuracy), 2) AS beginner_accuracy
FROM classification;
```

```
SELECT
    lesion_type,
    ROUND(AVG(expert_accuracy), 2) AS expert_accuracy,
    ROUND(AVG(beginner_accuracy), 2) AS beginner_accuracy
FROM classification
GROUP BY lesion_type;
```

```
SELECT * FROM classification
WHERE beginner_accuracy > expert_accuracy;
```

-- 7. The Metric Value provided does not seem significant enough to predict lesion type

```
SELECT ROUND(STDDEV(metric_value), 2) AS stddev_numeric_value FROM measurement;
```

```
SELECT m.metric_value, c.expert_accuracy FROM measurement m
JOIN classification c ON m.lesion_id = c.lesion_id
WHERE metric_value IS NOT NULL;
```

```
SELECT CORR(m.metric_value, c.expert_accuracy) AS corr_metric_expert_accuracy FROM
measurement m
JOIN classification c ON m.lesion_id = c.lesion_id;
```

```
SELECT CORR(m.metric_value, c.beginner_accuracy) AS corr_metric_beginner_accuracy
FROM measurement m
JOIN classification c ON m.lesion_id = c.lesion_id;
```

-- 8. What is the average actual status of a polyp:

```
SELECT
    ground_truth AS lesion_type,
    COUNT(lesion_id) AS total_count,
    ROUND(
        (COUNT(lesion_id) * 100) / (SELECT COUNT(*) FROM lesion), 2
    ) AS percentage_total
FROM lesion
GROUP BY ground_truth
ORDER BY total_count DESC;
```

-- 12. Finding cases of misdiagnosis'

```
SELECT l.ground_truth, c.avg_expert_claim, COUNT (*) AS cases FROM classification c
JOIN lesion l ON c.lesion_id = l.lesion_id
WHERE l.ground_truth <> c.avg_expert_claim
GROUP BY l.ground_truth, c.avg_expert_claim
ORDER BY cases;
```

-- 13. Determining risk per lesion

```
SELECT lesion_id, avg_expert_claim, expert_accuracy,
    CASE
        WHEN expert_accuracy > 70 THEN 'HIGH RISK'
        WHEN expert_accuracy > 50 AND expert_accuracy < 69 THEN 'MODERATE
RISK'
        ELSE 'LOW RISK'
    END AS risk_level
FROM classification;
```

```
SELECT lesion_id, avg_beginner_claim, beginner_accuracy,
    CASE
        WHEN beginner_accuracy > 70 THEN 'HIGH RISK'
        WHEN beginner_accuracy > 50 AND beginner_accuracy < 69 THEN
'MODERATE RISK'
        ELSE 'LOW RISK'
    END AS risk_level
FROM classification;
```


-- Total counts of each lesion (I already cleaned the histology class earlier to help categorize complex names given to different lesions by using LIKE/ILIKE)

```
SELECT histology_class AS lesion_type, COUNT (*) AS total_lesions FROM lesion_outcome  
GROUP BY histology_class  
ORDER BY total_lesions DESC;
```

-- 15. Finding where histological sites are commonly found in colorectal tract

```
SELECT  
    i.site AS lesion_site,  
    i.histology_extended AS histology,  
    COUNT(i.unique_object_id) AS total_lesions  
FROM lesion_info i  
GROUP BY i.site, i.histology_extended  
ORDER BY total_lesions DESC, lesion_site ASC;
```

-- BONUS. Might be helpful later for visualization

```
SELECT "Crohn's disease", crc_percentage FROM crc_clean  
ORDER BY crc_percentage DESC;
```

```
[4]: df_final = pd.read_csv('/Users/cullen/Desktop/DISC691_AppliedDataScience&Analyti/FinalDataFrames/df_final.csv')
```

```
df_final
```

```
[4]:
```

	Lesion_ID	Type	Quality_Metric	Numeric_Value	Lesion_Type
0	adenoma_1	3	1	138.120	Adenoma
1	adenoma_8	3	2	80.415	Adenoma
2	adenoma_9	3	2	106.160	Adenoma
3	adenoma_10	3	1	148.730	Adenoma
4	adenoma_11	3	2	109.130	Adenoma
...	...	...	...	...	...
71	serrated_5	2	1	114.590	Serrated
72	serrated_6	2	1	163.680	Serrated
73	serrated_7	2	1	180.110	Serrated
74	serrated_8	2	2	96.852	Serrated
75	serrated_9	2	1	93.569	Serrated

76 rows x 5 columns

```
[5]: # Lesions with the lowest Type code are associated with slightly higher numeric feature values.  
# These are usually resected. perhaps a high numeric_value can suggest a need for resection.
```

```
corr_TypeQualityNumeric = df_final[['Type', 'Quality_Metric', 'Numeric_Value']]  
corr3 = corr_TypeQualityNumeric.corr(method = 'spearman')
```

```
print(corr3, df_final['Type'].value_counts())  
df_final['Numeric_Value'].describe()
```

